AI FIREWALL: INTELLIGENT PROTECTION AGAINST PROMPT INJECTION AND MODEL ABUSE

A PROJECT REPORT

Submitted by

SRI RAM U	**2116231901053**
GURU SAI CHARAN D	**2116231901501**

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled **"AI FIREWALL: INTELLIGENT PROTECTION AGAINST PROMPT INJECTION AND MODEL ABUSE"** is the bonafide work of **"SRI RAM U. (2116231901053), SAI CHARAN D. (2116231901501)"** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. P. Tamilselvi, M.E., Ph.D.,

HEAD OF THE DEPARTMENT

Professor

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

SIGNATURE

Ms. Swathi Lakshmi S, M.Tech.,

SUPERVISOR

Assistant Professor

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on 06/10/2026

Internal Examiner

External Examiner

ABSTRACT

Large Language Models (LLMs) are now embedded in chat assistants, document-processing tools, Retrieval-Augmented Generation (RAG) applications, and enterprise automation. Because these systems accept natural-language instructions, they are exposed to prompt injection, jailbreak attacks, malicious instructions hidden in documents, leakage of sensitive information, and other forms of model abuse. Defences based only on static keywords or fixed patterns are easy to bypass and do not keep pace with evolving attacks.

This project proposes AI Firewall, a multi-layer security framework that protects an LLM application before and after the model runs. User inputs are examined by a keyword filter, a regular-expression pattern filter, and an on-device AI classifier. A risk scoring engine combines the three results into a low, medium, or high risk level, and high-risk requests are blocked before they reach the LLM.

For RAG applications, uploaded documents are screened before they are added to the retrieval index through text extraction, OCR, hidden-instruction detection, and content cleaning. After the LLM responds, an output firewall checks the response for sensitive information leakage, dangerous instructions, jailbreak indicators, and policy violations. Attacks detected only by the classifier are logged and used to propose new keywords and patterns, which are added to the fast filters only after human review and approval.

The proposed framework integrates input filtering, secure RAG document screening, semantic classification, risk-based decisions, output filtering, and continuous lexicon learning. The prototype will be evaluated using detection, latency, document-screening, and output-filtering metrics.

ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our Chairman **Mr. S. Meganathan B.E., F.I.E.,** for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan Ph.D.,** for her enthusiastic motivation which inspired us a lot in completing this project and Vice Chairman **Mr. Abhay Shankar Meganathan B.E., M.S.,** for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college Principal **Dr. S.N. Murugesan M.E., Ph.D.,** and **Dr. P. Tamilselvi Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security)** and our project guide **Ms. Swathi Lakshmi S, Assistant Professor,** for her encouragement and guiding us throughout the project. We would like to thank our project coordinator **Mr. Benedict J.N., M.E., Assistant Professor (SG),** for his encouragement and guidance towards successful completion of this project and to our parents, friends, all faculty members and supporting staffs for their direct and indirect involvement in the successful completion of this project, and for their encouragement and support.

SRI RAM U
GURU SAI CHARAN D

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iii**
	ACKNOWLEDGEMENT	**iv**
	LIST OF TABLES	**vii**
	LIST OF FIGURES	**viii**
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	2
	1.3 OBJECTIVES	2
	1.4 SCOPE	3
	1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS	4
	1.6 SDG ALIGNMENT	5
2	**LITERATURE REVIEW**	**6**
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	6
	2.2 RESEARCH GAPS	9
	2.3 PROPOSED SYSTEM	9
3	**SYSTEM DESIGN**	**11**
	3.1 SYSTEM ARCHITECTURE	11
	3.2 MODULE DESIGN	12
	3.3 DATA FLOW / UML DIAGRAMS	13
	3.4 THREAT MODEL AND ATTACK SURFACE	16
	3.5 SECURITY CONTROLS	17
	3.6 DESIGN VALIDATION CHECKLIST	17
4	**SYSTEM DESCRIPTION**	**19**
	4.1 DATA COLLECTION	19
	4.2 DATA PREPROCESSING	19
	4.2.1 ALGORITHMS / METHODOLOGY	20
	4.2.2 RESULTS AND DISCUSSION	21
	4.3 ADDITIONAL ALGORITHMS	22
	4.4 DATA STORES AND LOG SCHEMA DESIGN	23
	4.5 TESTING STRATEGY	24

4.6 DEPLOYMENT CONSIDERATIONS 25

4.7 LIMITATIONS OF THE CURRENT PHASE 26

4.8 RESPONSIBLE USE AND PRIVACY CONSIDERATIONS 26

5 CONCLUSION 27

5.1 SUMMARY OF CONTRIBUTIONS 27

5.2 FUTURE WORK 27

REFERENCES 28

APPENDIX 29

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	NON-FUNCTIONAL REQUIREMENTS	4
1.2	ROLE-BASED PERMISSION MATRIX	4
1.3	SDG ALIGNMENT	5
2.1	SUMMARY OF REVIEWED LITERATURE (2019–2024)	8
3.1	THREAT MODEL SUMMARY	17
3.2	SECURITY CONTROLS MAPPING	17
3.3	RESEARCH GAP TO DESIGN ELEMENT MAPPING	18
4.1	ATTACK CATEGORIES AND DETECTION COVERAGE	20
4.2	ILLUSTRATIVE RISK SCORE CALCULATION (HYPOTHETICAL VALUES)	21
4.3	PLANNED EVALUATION METRICS	22
4.4	DATA STORES OF THE AI FIREWALL	24
4.5	ATTACK LOG RECORD FIELDS	24
4.6	TESTING STRATEGY SUMMARY	25
4.7	PLANNED FUNCTIONAL TEST CASES	25
4.8	DEPLOYMENT OPTIONS FOR THE AI FIREWALL	26
A.1	CONFIGURATION PARAMETERS OF THE AI FIREWALL	30
A.2	PROJECT RISK REGISTER	31
A.3	DECISION CODES	31
A.4	SAMPLE REVIEWER VIEW	32

LIST OF FIGURES

FIGURE	TITLE	PAGE NO.
3.1	SYSTEM ARCHITECTURE	11
3.2	INPUT FIREWALL PIPELINE	12
3.3	RAG SECURITY MODULE	13
3.4	USE CASE DIAGRAM	13
3.5	LEVEL-0 DATA FLOW DIAGRAM	14
3.6	LEVEL-1 DATA FLOW DIAGRAM	14
3.7	END-TO-END PROJECT WORKFLOW	15
3.8	REQUEST LIFECYCLE (SEQUENCE DIAGRAM)	15
3.9	TRUST BOUNDARIES AND THREAT POINTS OF THE AI FIREWALL	16
4.1	INPUT PREPROCESSING PIPELINE	20
4.2	RISK SCORING ENGINE	21
4.3	OUTPUT FILTERING FIREWALL	23
4.4	CONTINUOUS LEXICON LEARNING	23

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Large Language Models (LLMs) can understand and generate natural language, and they are now used in chatbots, coding assistants, customer support, document analysis, and enterprise information retrieval. Many applications also combine an LLM with Retrieval-Augmented Generation (RAG), in which relevant passages are retrieved from documents and databases and supplied to the model together with the user query. This allows the model to answer using information that was not part of its training data.

The flexibility of natural-language interaction also creates new security challenges. In a conventional program, instructions and data are kept separate. In an LLM application, the developer's instructions, the user's request, and any retrieved text all reach the model as plain text, and the model cannot reliably tell which part should be obeyed. An attacker can therefore write a prompt that tries to override the application's instructions, bypass its restrictions, extract confidential information, or make the model produce unintended content.

Two attack types are especially important. In prompt injection, the attacker inserts instructions that interfere with the original purpose of the application. In jailbreaking, carefully designed prompts try to bypass the safety restrictions of the model. RAG introduces a further attack surface: a malicious document may contain hidden instructions that are retrieved together with legitimate information and then influence the model's behaviour. Even when the input looks harmless, the response itself may leak private data or contain unsafe content.

To address these threats, the proposed project introduces an AI Firewall that surrounds the LLM application. Before a request is processed, the firewall checks it using fast keyword and pattern filters and, where necessary, an on-device AI classifier. A risk scoring engine decides whether the request is allowed, checked further, or blocked. Uploaded documents are screened before they enter the retrieval index, and the generated response is inspected before it is delivered. Newly detected attacks are logged and, after human review, used to improve the fast filters.

The proposed system therefore brings the following major functions into a single framework:

1. Multi-layer input screening (keyword, regex, and on-device AI classification).
2. Risk-based decision making before the LLM is invoked.
3. Secure screening of RAG documents and output filtering of LLM responses.
4. Attack logging with human-reviewed continuous lexicon learning.

1.2 PROBLEM STATEMENT

LLM-based applications face security threats that cannot be handled effectively by a single static filter. The following problems motivate this project:

- Prompt injection, in which attackers insert instructions that try to override the intended behaviour of the application.
- Jailbreak prompts designed to bypass the restrictions and safety mechanisms of the model.
- Malicious or hidden instructions inside documents that are uploaded to a RAG knowledge base.
- Novel and paraphrased attacks that contain no known keyword or predefined pattern.
- Leakage of private or confidential information in the generated response.
- Unsafe or policy-violating content in the model output.
- Attack techniques that change continuously, making fixed security rules insufficient over time.
- Dependence on external moderation services, which adds latency and may send user data outside the local environment.

The project therefore focuses on a security layer in which user inputs, RAG documents, and model outputs are all inspected, decisions are based on a combined risk score, and the detection rules improve over time under human supervision.

1.3 OBJECTIVES

The overall objective of this project is to design and develop a multi-layer AI Firewall that reduces the risk of prompt injection and model abuse in LLM applications while keeping detection fast and, where possible, local. The specific objectives of this work are:

1. To design a multi-layer security architecture for LLM applications.
2. To detect prompt injection attacks before they reach the LLM.
3. To identify known jailbreak and malicious prompt patterns.
4. To detect subtle and paraphrased attacks using an on-device AI classifier.
5. To secure RAG documents before they are added to the retrieval index.
6. To calculate a risk score by combining the results of multiple security layers.
7. To block high-risk requests before LLM processing.
8. To inspect LLM responses for unsafe or sensitive content.
9. To maintain attack logs for security analysis and audit.

10. To continuously improve the security lexicon using newly detected attack patterns after human review.

1.4 SCOPE

The scope of this project covers the security of the interaction between users, external documents, and an LLM application. The system considers the following participants:

- End User
- Application Developer / Administrator
- Security Reviewer
- Knowledge-Base Contributor (uploads documents)
- LLM Application (protected system)

The system handles the following information during request processing:

- User prompts and uploaded files
- Extracted text and cleaned document content
- Layer scores, risk level, and the security decision
- Generated responses and safe default responses
- Attack logs, lexicon, pattern set, and configuration parameters
- Reviewer decisions and audit records

The current phase focuses on the design of a three-layer input firewall, a RAG security module, an output filtering firewall, and a reviewed learning loop. The following are outside the scope of the current phase:

- Training, fine-tuning, or modifying the underlying LLM
- Attacks on model weights or training data, such as data poisoning and model extraction
- Attacks on the host operating system, network, or hosting infrastructure
- Audio and video inputs; images are handled only through OCR-based text extraction
- A guarantee that every possible attack will be detected

The AI Firewall reduces risk but is not a replacement for other security practices. Beyond these functional requirements, the system is also governed by non-functional requirements that shape its design, summarized in the table below.

Requirement	Description	Design Implication
Performance	Keyword and regex checks should add little delay for legitimate requests; deeper analysis is used when needed	Fast layers run first; classifier invoked by policy
Privacy	Request content should not have to leave the local environment for security analysis	On-device classifier
Modularity	Each security layer should be replaceable without changing the others	Independent L1, L2, L3 modules
Configurability	Weights, thresholds, and the safe default response should be adjustable	Parameters kept in a configuration store
Reliability	The firewall should fail safely if a layer cannot produce a result	Conservative fallback policy
Maintainability	Lexicon and patterns should be updatable without redeploying the whole system	Reviewed learning loop with versioned updates

Table 1.1 Non-Functional Requirements

1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS

Because an LLM application can be manipulated through its inputs, its documents, and its outputs, the security layer itself must be carefully controlled. The following security requirements guide the design of the proposed system.

1. Authentication and Access Control

Administrative and review functions must be available only to authenticated and authorized users. Ordinary users cannot change rules, thresholds, or the lexicon.

2. Role-Based Authorization

Access to system functions is controlled according to the participant's role.

Role	Main Permission
End User	Submit prompts, upload documents, receive safe responses
Security Reviewer	Review candidate patterns; approve, reject, modify, or defer them
Administrator	Configure weights and thresholds; manage rules; view logs
Lexicon Learner (system component)	Propose candidate keywords and patterns only; cannot deploy them
RAG Security Module (system component)	Only component allowed to write to the retrieval index

Table 1.2 Role-Based Permission Matrix

3. Confidentiality

System instructions, private documents, and sensitive data must not be exposed through prompts or generated responses.

4. Integrity of Instructions

User input and retrieved documents must not be able to alter the intended instructions and behaviour of the LLM application.

5. Input Inspection

Every prompt and every uploaded document is inspected before it can influence the LLM.

6. Fail-Safe Behaviour

When a request or response is judged unsafe, the system blocks it or replaces it with a safe default instead of passing it on.

7. Auditability

Each flagged event records the request, detection layer, risk score, attack category, timestamp, and decision to support analysis and audit.

8. Controlled Change

New patterns learned from attacks are deployed only after a human reviewer has approved them, which prevents harmful patterns from entering the filters.

9. Data Privacy

Classification is performed on-device, so request content does not have to be sent to an external security service.

1.6 SDG ALIGNMENT

This project can be associated with the following United Nations Sustainable Development Goals, reflecting both its technical contribution and its intended real-world impact.

S.No.	SDG No. & Goal	Relevant SDG Target	Project Contribution / Alignment
1	SDG 9 – Industry, Innovation and Infrastructure	Targets 9.1 and 9.5	Provides a security layer that helps make AI-enabled digital infrastructure more trustworthy and resilient.
2	SDG 16 – Peace, Justice and Strong Institutions	Target 16.10	Protects users and organizations from misuse of LLM applications and from leakage of sensitive information.

Table 1.3 SDG Alignment

CHAPTER 2
LITERATURE REVIEW

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

Existing protection for LLM applications can be broadly classified into keyword-based filtering, regex-based detection, external moderation services, basic RAG pipelines, and output-only protection. Each approach provides a different level of security and has its own limitations.

2.1.1 Keyword-Based Filtering

Keyword filtering compares the input against a predefined list of suspicious terms. It is fast, simple, and easy to implement. However, it cannot understand context, can be bypassed through paraphrasing, requires continuous updates, and may produce false positives when a harmless request contains a listed word.

2.1.2 Regex-Based Detection

Regular expressions detect known attack structures and predefined patterns. They offer fast matching and low computational overhead and suit known attack formats. However, they cannot reliably detect semantic attacks, attackers can change the structure of a prompt to avoid a pattern, and every pattern must be defined in advance.

2.1.3 External Moderation APIs

Some applications send prompts and responses to an external security or moderation service. This provides semantic judgement without maintaining a local model, but it requires network communication, can increase response latency, may move user data out of the local environment, and depends on a third party. It may therefore be unsuitable for privacy-sensitive environments.

2.1.4 Basic RAG Systems

A conventional RAG system retrieves information from a vector database and sends it to the LLM [11]. If a stored document contains malicious instructions, those instructions can become part of the model context. This indirect route, described as indirect prompt injection [3], means that securing only the user prompt is not enough.

2.1.5 Output-Only Protection

Some systems inspect only the generated response. This is insufficient because the malicious request has already reached the model by the time the response is checked.

2.1.6 Summary of Security Approaches in Existing Systems

Existing approaches commonly use keyword lists, patterns, moderation services, safeguard models, and output checks. However, the level of integration between input filtering, document screening, semantic classification, output filtering, and controlled rule updates

varies across existing systems. This provides the basis for identifying the research gaps addressed by the proposed system.

2.1.7 Review of Recent Literature on Prompt Injection and LLM Security (2019–2024)

Recent studies show that LLM applications face security risks that differ from those of conventional software. The OWASP list of risks for LLM applications [1] ranks prompt injection and sensitive information disclosure among the most important problems. Perez and Ribeiro [2] showed how handcrafted inputs can hijack the goal of a prompt or cause a model to reveal its hidden instructions, and Liu et al. [4] proposed a black-box technique for injecting instructions into deployed LLM-integrated applications.

Greshake et al. [3] demonstrated indirect prompt injection, in which instructions hidden in retrieved content steer the behaviour of an application. Studies of jailbreaking [5], [6] trace failures of safety training to competing objectives and mismatched generalization and show that automatically generated suffixes can transfer between models.

On the defence side, Alon and Kamfonas [7] and Jain et al. [8] examined perplexity-based detection, paraphrasing, and retokenization as baseline countermeasures, and Hines et al. [9] proposed transforming untrusted text so that the model can better distinguish it from instructions. Llama Guard [10] uses a fine-tuned language model to classify prompts and responses against a risk taxonomy. For implementation, distilled transformers such as DistilBERT [12] and MiniLM [13] and vector-search libraries such as FAISS [14] provide compact building blocks for an on-device classifier and a local retrieval index.

However, the reviewed literature treats these concerns mostly one at a time. Therefore, the proposed system combines fast lexical filters, a local semantic classifier, document screening, output filtering, and reviewed rule updates in a single framework.

Ref.	Focus Area	Relevance to Proposed System
[1]	OWASP Top 10 for LLM applications	Common vocabulary for the threats considered
[2]	Prompt-based attack techniques	Motivates filtering of user input before the LLM
[3]	Indirect prompt injection	Motivates screening of documents before indexing
[4]	Injection into LLM-integrated applications	Shows real applications can be attacked through prompts
[5], [6]	Why safety training fails; transferable adversarial suffixes	Protection should not rely on the model alone
[7], [8]	Perplexity-based detection and baseline defences	Complementary detectors; limited to specific attack families
[9]	Defending against indirect injection	Reinforces the need to treat untrusted text carefully
[10]	LLM-based input–output safeguard	Reference point for guard models and for checking inputs and outputs
[11]	Retrieval-Augmented Generation	Defines the pipeline that the RAG module protects
[12], [13]	DistilBERT and MiniLM	Candidate models for the on-device classifier
[14]	FAISS similarity search	Local vector index for the screened knowledge base

Table 2.1 Summary of Reviewed Literature (2019–2024)

2.1.8 Comparative Discussion

The reviewed studies [2]–[6] show that attacks can arrive through the prompt, through retrieved content, and through the model's own safety weaknesses. Studies of defences [7]–[10] show that detectors are largely complementary, and several of them target one attack family rather than the whole pipeline. This highlights the need for several cooperating layers.

Another common observation is that semantic models such as safeguard classifiers give stronger coverage than fixed rules, but at a higher computational cost and, for hosted services, with data leaving the local environment [10]. The proposed system follows a layered approach: cheap checks run first, and a compact local model is used for deeper analysis.

Overall, the comparison identifies two major gaps: integrated screening of both user input and RAG documents, and a controlled way of keeping the fast filters up to date. These form the main focus of the proposed system.

2.2 RESEARCH GAPS

From the literature reviewed above, six specific gaps motivate the design of the proposed system.

Gap 1 — Reliance on a Single Detection Technique

Keyword and pattern filters are fast but easily evaded by rewording. The proposed system combines three input layers.

Gap 2 — Documents Not Screened

Many defences examine only the user prompt, although instructions can also enter through retrieved documents. The proposed system screens documents before indexing.

Gap 3 — Privacy and Dependence on External Services

Hosted moderation sends every request outside the local environment. The proposed system performs classification on-device.

Gap 4 — No Graded Decision

Simple filters make a yes/no decision. The proposed system combines several scores into a low, medium, or high risk level.

Gap 5 — Input-Only or Output-Only Protection

Protection at one end of the pipeline leaves the other end exposed. The proposed system applies an input firewall and an output firewall.

Gap 6 — Static Rules That Do Not Evolve

Attack techniques change continuously, but rule sets are updated slowly. The proposed system learns from logged attacks under human review.

2.3 PROPOSED SYSTEM

2.3.1 Overview of the Proposed System

The proposed AI Firewall acts as a security layer around an LLM application. A user prompt first enters the input firewall, where the keyword filter (L1), the regex filter (L2), and, when required, the on-device AI classifier (L3) each produce a score. The risk scoring engine combines the scores. High-risk requests are blocked, and approved requests are passed to the LLM and RAG component.

Uploaded documents follow a separate path through the RAG security module, which extracts text, performs OCR where needed, detects hidden instructions, and cleans the content before it is embedded and added to the retrieval index. The generated response is inspected by the output filtering firewall before it is delivered. Detected attacks are logged and reviewed so that the fast filters can be improved.

2.3.2 Objectives of the Proposed System

The proposed system aims to:

- Detect malicious prompts before they reach the LLM.
- Protect the knowledge base from malicious documents.
- Detect subtle and paraphrased attacks locally without an external API.
- Make graded, risk-based security decisions.
- Inspect responses for leakage and unsafe content.
- Maintain an auditable record of security events.
- Improve detection over time through reviewed lexicon learning.
- Develop a practical prototype using available technologies.

2.3.3 Advantages of the Proposed System

1. Multi-layer security — several independent layers provide defence in depth, so a miss by one layer can be caught by another.
2. Faster detection — keyword and regex filters quickly detect known attacks.
3. Intelligent detection — the on-device classifier can identify subtle and paraphrased attacks.
4. Secure RAG — documents are screened before they enter the retrieval database.
5. Privacy — analysis can be performed without sending every request to an external service.
6. Risk-based decisions — requests are classified according to their security risk.
7. Output protection — unsafe responses can be detected before reaching the user.
8. Continuous improvement — the firewall learns new keywords and patterns through a reviewed learning loop.

CHAPTER 3
SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The proposed system is organized into the following layers: the Input Source, the Input Firewall (L1, L2, and L3), the Risk Scoring Engine, LLM and RAG Processing, the Output Filtering Firewall, and the Learning and Logging layer. A separate RAG Security Module protects the retrieval index.

The Input Source accepts text prompts and uploaded files. The Input Firewall checks each prompt using the keyword filter, the regex filter, and the on-device classifier. The Risk Scoring Engine combines the layer scores and decides whether to allow, check further, or block. Only approved requests reach the LLM, which uses the screened retrieval index when RAG is enabled. The Output Filtering Firewall inspects the response. The Lexicon Learner and the data stores (attack logs, lexicon, configuration, and audit records) support continuous improvement.

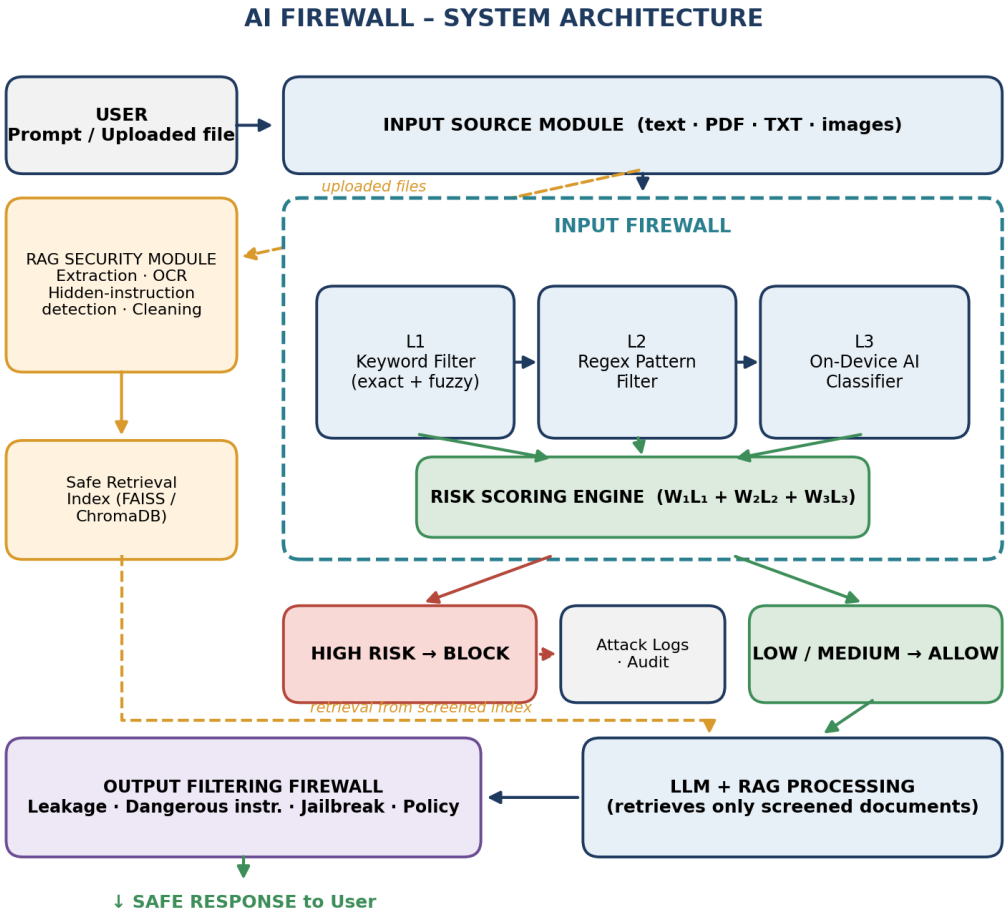


Fig. 3.1 System Architecture

3.2 MODULE DESIGN

The system is divided into functional modules for input handling, document screening, input analysis, risk scoring, LLM processing, output filtering, learning, and logging.

Module 1 — Input Source

Accepts user text, PDF files, TXT files, and images and forwards each input to the appropriate security pipeline.

Module 2 — RAG Security

Screens uploaded documents through text extraction, OCR, hidden-instruction detection, and content cleaning before indexing.

Module 3 — Keyword Filtering (L1)

Compares the normalized request with the security lexicon using exact and fuzzy matching and returns the L1 score.

Module 4 — Regex Pattern Filtering (L2)

Matches the request against predefined patterns for instruction overrides, known jailbreak formats, and suspicious payload structures and returns the L2 score.

Module 5 — On-Device AI Classification (L3)

Uses a lightweight local model to detect subtle, paraphrased, and novel attacks and returns the L3 score.

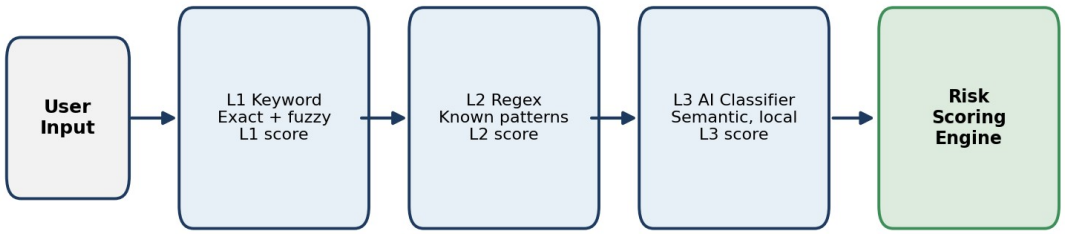


Fig. 3.2 Input Firewall Pipeline

Module 6 — Risk Scoring Engine

Combines the L1, L2, and L3 scores using configurable weights and classifies the request as low, medium, or high risk.

Module 7 — LLM and RAG Processing

Passes only approved requests to the LLM and retrieves context only from the screened retrieval index.

Module 8 — Output Filtering Firewall

Checks the generated response for private data leakage, dangerous instructions, jailbreak indicators, and policy violations.

Module 9 — Continuous Lexicon Learning

Extracts candidate keywords and patterns from logged attacks and submits them for human review.

Module 10 — Attack Logging and Audit

Records flagged requests, detection layer, risk score, attack category, timestamp, decision, and reviewer actions.



Fig. 3.3 RAG Security Module

3.3 DATA FLOW / UML DIAGRAMS

This section presents the use case, data flow, workflow, and sequence diagrams to describe the system's users, processing steps, and component interactions. The data flow consists of input collection, document screening, fast filtering, classification, risk scoring, LLM processing, output filtering, and logging.

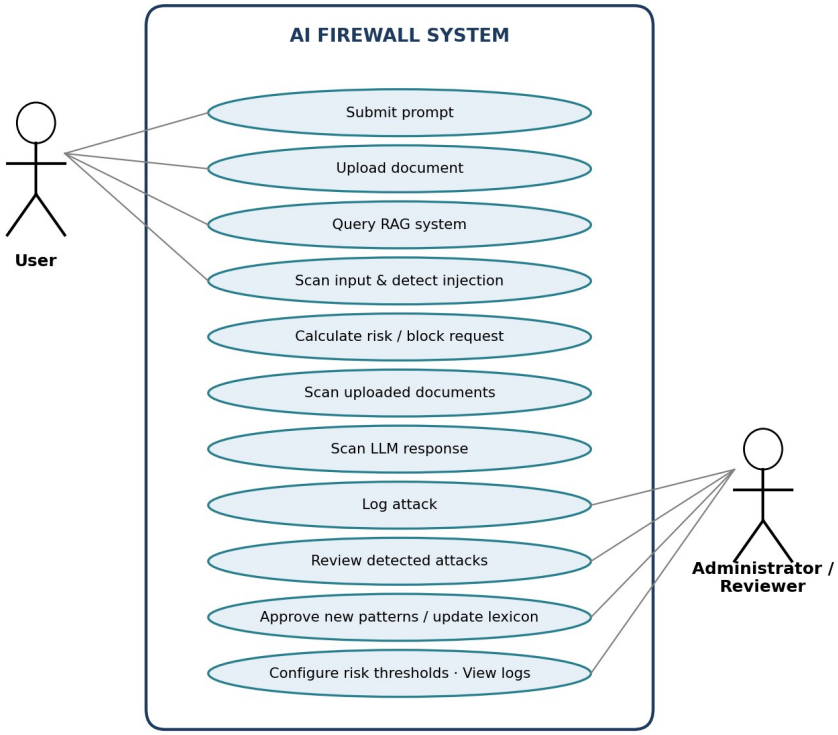


Fig. 3.4 Use Case Diagram

Fig. 3.4 shows the two types of actors and their main use cases. Users submit prompts and documents and receive safe responses, while administrators and reviewers review attacks, approve patterns, configure thresholds, and view logs.

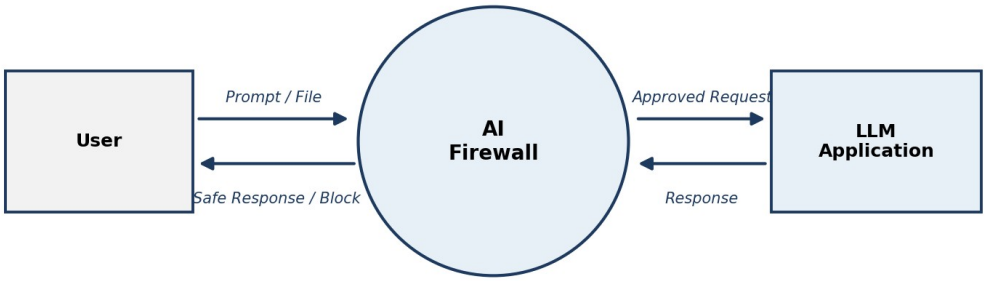


Fig. 3.5 Level-0 Data Flow Diagram

Fig. 3.5 shows the AI Firewall as a single process between the user and the LLM application. Only approved requests are passed on, and a safe response or block message is returned to the user.

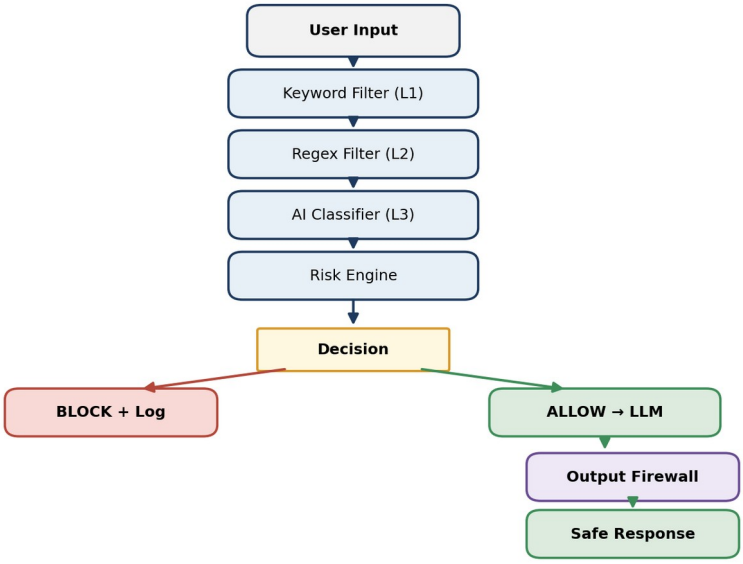


Fig. 3.6 Level-1 Data Flow Diagram

Fig. 3.6 expands the firewall into the keyword filter, regex filter, AI classifier, and risk engine. The decision either blocks the request or allows it to reach the LLM, after which the output firewall inspects the response.

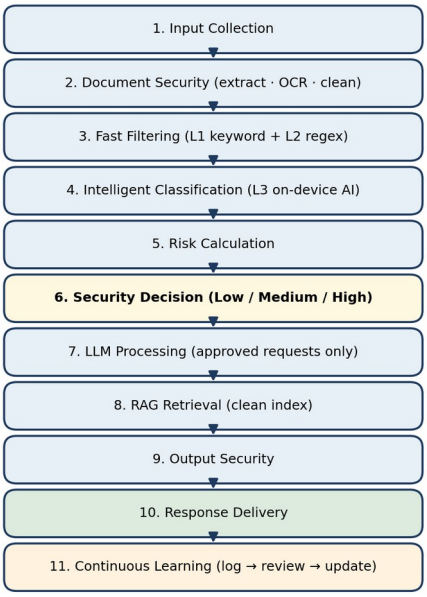


Fig. 3.7 End-to-End Project Workflow

Fig. 3.7 shows the complete workflow, from input collection and document security to risk calculation, LLM processing, output security, response delivery, and continuous learning.

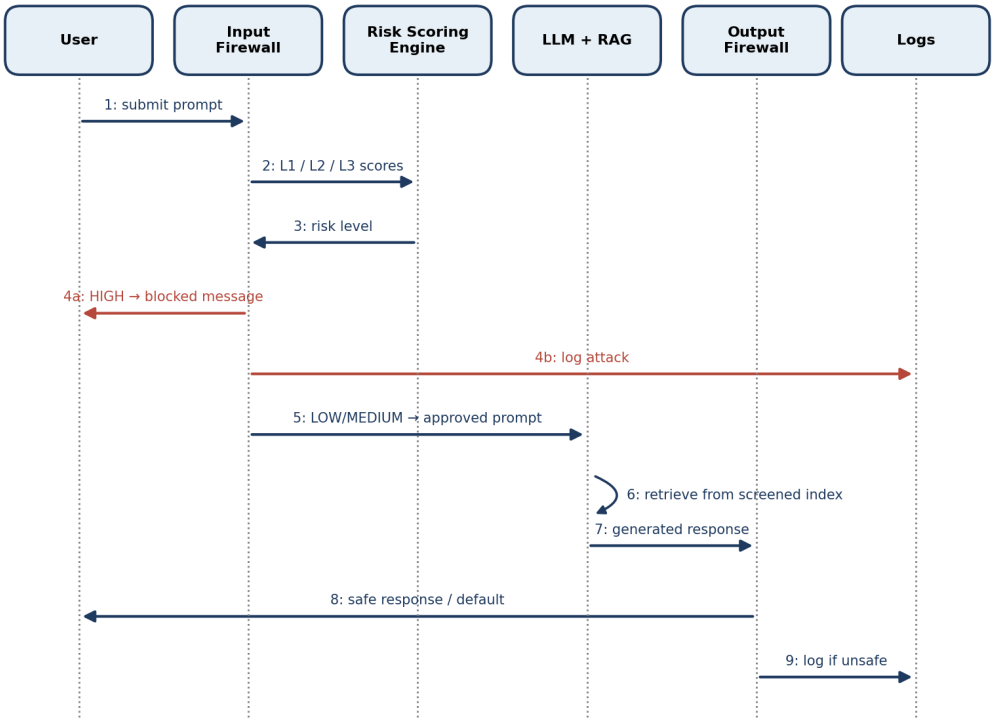


Fig. 3.8 Request Lifecycle (Sequence Diagram)

Fig. 3.8 shows the message flow for a single request. A high-risk result produces a blocked message and a log entry; otherwise the prompt reaches the LLM, retrieval uses the screened index, and the output firewall vets the response before it is returned.

3.4 THREAT MODEL AND ATTACK SURFACE

The assets to be protected are the hidden instructions and configuration of the LLM application, the contents of the knowledge base, any confidential data the model can reach, and the integrity of the security rules. The main threats identified in the proposed system are:

1. **Direct Prompt Injection / Jailbreak** – Users may craft prompts that try to override instructions or bypass restrictions.
2. **Malicious Document** – Uploaded documents may contain hidden instructions that enter the model context through retrieval.
3. **Sensitive Data Leakage** – The response may reveal private or confidential information.
4. **Unsafe Output** – The response may contain harmful or policy-violating content.
5. **Filter Evasion** – Attackers may paraphrase or obfuscate input to avoid known keywords and patterns.
6. **Lexicon Poisoning** – Attackers may try to cause harmful or misleading entries to be learned.
7. **Denial through Filter Abuse** – Crafted input may try to overload the pattern filter or the classifier.

Figure 3.9 shows the trust boundaries of the system and the numbered threat points, and Table 3.1 summarizes the attack surface for each threat.

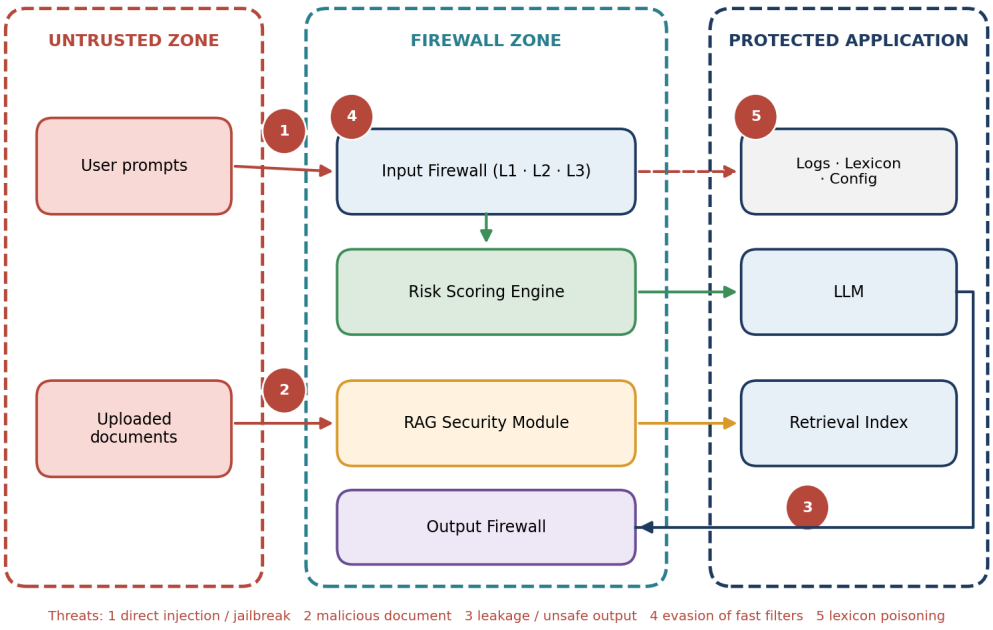


Fig. 3.9 Trust Boundaries and Threat Points of the AI Firewall

Threat	Primary Attack Surface	Potential Impact
Direct injection / jailbreak	User prompt	Application instructions overridden
Malicious document	Uploaded file and retrieval index	Hidden instructions reach the LLM context
Sensitive data leakage	LLM response	Confidential information exposed
Unsafe output	LLM response	Harmful instructions delivered to the user
Filter evasion	L1 and L2 filters	Known rules fail to match a novel attack
Lexicon poisoning	Learning loop	False positives or weakened detection
Filter abuse	Regex and classifier layers	Service slowdown

Table 3.1 Threat Model Summary

3.5 SECURITY CONTROLS

The following controls are proposed to mitigate the threats identified in Section 3.4.

Threat	Security Control
Direct injection / jailbreak	L1 keyword and L2 regex filters, L3 on-device classifier, risk scoring and blocking
Malicious document	RAG Security Module (hidden-instruction detection and cleaning); retrieval only from the screened index
Sensitive data leakage	Output Filtering Firewall (private data check)
Unsafe output	Output Filtering Firewall (dangerous instruction and policy checks); safe default response
Filter evasion	On-device classifier and continuous lexicon learning
Lexicon poisoning	Mandatory human review and approval; audit log of decisions
Filter abuse	Linear-time regex engine, input length limits, and chunking
Audit issues	Structured attack logs with timestamp, layer, score, and decision

Table 3.2 Security Controls Mapping

3.6 DESIGN VALIDATION CHECKLIST

As a closing check on the design presented in this chapter, Table 3.3 maps each research gap identified in Section 2.2 to the specific module or design element that addresses it, so that the connection between motivation and design is explicit rather than implied.

Research Gap (Section 2.2)	Addressed By
Gap 1 — Reliance on a single detection technique	Modules 3, 4, and 5 (L1, L2, L3)
Gap 2 — Documents not screened	Module 2 (RAG Security)
Gap 3 — Privacy and external dependence	Module 5 (On-device classifier)
Gap 4 — No graded decision	Module 6 (Risk Scoring Engine)
Gap 5 — Input-only or output-only protection	Input Firewall and Module 8 (Output Filtering Firewall)
Gap 6 — Static rules	Modules 9 and 10 (Lexicon Learning, Logging)

Table 3.3 Research Gap to Design Element Mapping

CHAPTER 4
SYSTEM DESCRIPTION

4.1 DATA COLLECTION

The system collects and handles data from different parts of the LLM pipeline, including:

Prompt Data:

- User text, session identifier, and timestamp.

Document Data:

- Uploaded PDF, TXT, and image files, extracted text, OCR output, and cleaned content.

Response Data:

- Generated response text and the output-firewall verdict.

Security Knowledge:

- Keyword lexicon, regex pattern set, and classifier model.

Log Data:

- Flagged requests, detection layer, scores, attack category, and decision.

Configuration Data:

- Layer weights, risk thresholds, safe default response, and policy settings.

4.2 DATA PREPROCESSING

Before any detector runs, the submitted input passes through a sequence of preprocessing operations designed to defeat simple obfuscation. The normalized text is used for detection only; the original text is preserved in the log for review.

Step 1 — Unicode Normalization

Converts visually equivalent characters to a standard form so that look-alike characters do not defeat matching.

Step 2 — Case Folding

Treats upper-case and lower-case forms of a word as the same.

Step 3 — Whitespace Collapsing

Removes repeated spaces and line breaks inserted to split a keyword.

Step 4 — Removal of Invisible Characters

Strips zero-width and other non-printing characters that can hide text from simple filters.

Step 5 — Detection of Encoded Segments

Identifies segments that appear to be encoded and passes the decoded form to the detection layers.

Step 6 — Chunking of Long Text

Splits long prompts or documents into segments so that each segment can be analysed by the classifier.

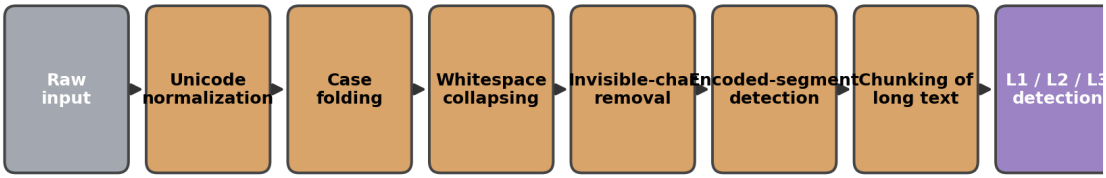


Fig. 4.1 Input Preprocessing Pipeline

4.2.1 ALGORITHMS / METHODOLOGY

A. Request Screening Algorithm

The system normalizes the request, computes the keyword score L1 and the regex score L2, and invokes the classifier according to a configured policy to obtain L3. The three scores are combined by the risk scoring engine. High-risk requests are blocked and logged, medium-risk requests receive additional checking, and low-risk requests are passed to the LLM. Table 4.1 shows which layer is the primary and which is the supporting defence for each attack category.

Attack Category	Primary Layer	Supporting Layer
Instruction override phrases	L1 / L2	L3
Role-play and persona-based jailbreaks	L2 / L3	L1
System prompt extraction requests	L2	L3, output firewall
Paraphrased or novel manipulation	L3	Lexicon learning
Obfuscated or encoded payloads	L2 / L3	Preprocessing
Hidden instructions in documents	RAG Security Module	Output firewall
Sensitive data in responses	Output firewall	Audit logging

Table 4.1 Attack Categories and Detection Coverage

B. Risk Scoring Method

Each layer returns a score normalized between 0 and 1. The total risk score is the weighted sum

$$R = W_1L_1 + W_2L_2 + W_3L_3, \quad with\ W_1 + W_2 + W_3 = 1$$

where L_1, L_2, and L_3 are the keyword, regex, and classifier scores and W_1, W_2, W_3 are configurable weights. Two thresholds, T_low and T_high, define the decision:

$$Decision = Allow\ if\ R < T_low;\ \ Additional\ check\ if\ T_low \leq R < T_high;\ \ Block\ if\ R \geq T_high$$



Fig. 4.2 Risk Scoring Engine

Table 4.2 illustrates the method using hypothetical values: W = (0.2, 0.3, 0.5), T_low = 0.35, and T_high = 0.70. These values are chosen only for explanation. They are not measured results, and the actual values will be tuned on evaluation data.

Scenario	L1	L2	L3	Total Risk	Level
Ordinary, harmless question	0.00	0.00	0.05	0.03	Low
Harmless question mentioning a security term	0.60	0.00	0.15	0.20	Low
Paraphrased manipulation, no known keyword	0.00	0.00	0.90	0.45	Medium
Known instruction-override formulation	1.00	1.00	0.95	0.98	High

Table 4.2 Illustrative Risk Score Calculation (Hypothetical Values)

The example shows three design points. A harmless prompt that merely contains a suspicious word is not blocked, because the other layers do not support the suspicion. A paraphrased attack that no keyword or pattern matches still receives a significant classifier score and falls in the medium range, where additional checking is applied. A prompt that triggers all three layers is blocked before it reaches the LLM.

C. Document Screening Method

An uploaded document is processed by text extraction (with OCR for images and scanned pages), detection of hidden or embedded instructions, and content cleaning. Severe findings cause the document to be rejected; otherwise only the cleaned content is embedded and added to the safe retrieval index. Because screening is done once at upload time, its cost is paid outside the request path.

4.2.2 RESULTS AND DISCUSSION

As this is a Phase I report, this section focuses on the design and the planned evaluation rather than on assumed performance results. The proposed workflow covers input

screening, document screening, risk scoring, LLM processing, output filtering, and reviewed learning.

The system ensures that user inputs and documents are inspected before they can influence the LLM, and that responses are inspected before they reach the user. Functional testing will cover the blocking of known attacks, the handling of paraphrased attacks, document screening, output filtering, and reviewer actions. Performance evaluation will consider detection latency and end-to-end processing time.

Module	**Evaluation Metrics**
Input Firewall (L1, L2, L3)	Accuracy, precision, recall, F1 score, false positive rate, false negative rate, detection latency
Risk Scoring Engine	Decision correctness at each risk level, sensitivity to weights and thresholds
RAG Security	Detection rate of hidden instructions, rejection rate of clean documents, document processing time
Output Firewall	Detection rate for leakage and unsafe content, false positive rate on safe responses
Lexicon Learning	Number of candidates proposed, approval rate, change in detection after update
System (End-to-End)	End-to-end response time and overall functional reliability

Table 4.3 Planned Evaluation Metrics

No final numerical performance values are reported at this stage, as the complete experimental evaluation has not yet been conducted. These values will be measured and reported in the subsequent phase of this project once implementation and testing have been carried out.

4.3 ADDITIONAL ALGORITHMS

D. Output Filtering Algorithm

The generated response is checked for private data leakage, dangerous instructions, jailbreak indicators, and content-policy violations. If all checks pass, the response is delivered. If any check fails, the response is replaced by a safe default message and the event is logged. If the check cannot be completed, the response is withheld, in keeping with the fail-safe principle.

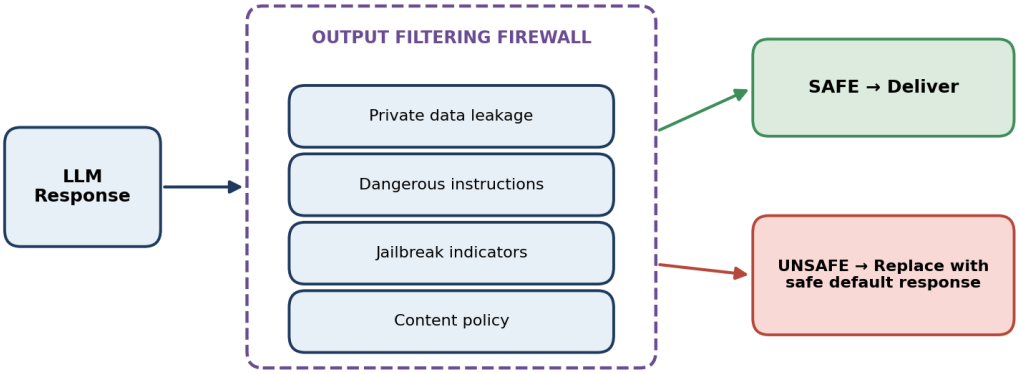


Fig. 4.3 Output Filtering Firewall

E. Lexicon Learning Algorithm

Detections made only by the classifier are the most informative, because they show what the fast layers miss. Such events are logged, and candidate keywords and patterns are extracted and de-duplicated. A reviewer approves, rejects, modifies, or defers each candidate. Approved entries are added to L1 and L2 and deployed with the next filter update, and every decision is recorded in the audit log. Because the learner can only propose entries, an attacker who provokes the classifier cannot rewrite the rules.

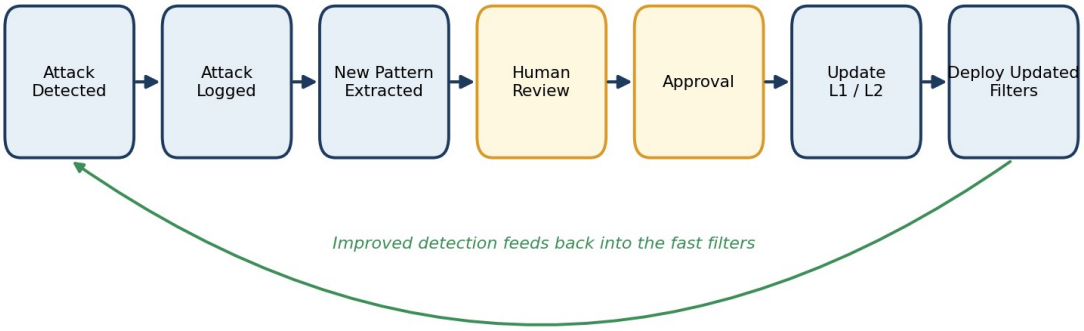


Fig. 4.4 Continuous Lexicon Learning

4.4 DATA STORES AND LOG SCHEMA DESIGN

The system maintains several separate data stores so that different access rules can be applied to each. For example, the retrieval index is written only by the RAG Security Module, while the lexicon is changed only after reviewer approval.

Data Store	Contents	Written By
Attack log	Flagged requests, detection layer, risk score, attack category, timestamp, decision	Input and output firewalls
Security lexicon	Approved keywords and fuzzy-match entries used by L1	Lexicon learner after approval
Pattern set	Approved regular-expression patterns used by L2	Lexicon learner after approval
Configuration	Weights, thresholds, safe default response, other parameters	Administrator
Retrieval index	Embeddings of screened and cleaned documents	RAG Security Module
Audit records	Reviewer identity and approval or rejection of patterns	Review interface

Table 4.4 Data Stores of the AI Firewall

Each attack-log record contains the fields listed in Table 4.5. Storing the individual layer scores together with the decision makes it possible to explain afterwards why a request was handled in a particular way.

Field	Description
timestamp	Date and time of the event
flagged_request	The request or document that was flagged
detection_layer	Layer that flagged the event (L1, L2, L3, RAG module, or output firewall)
risk_score	Total risk score computed by the Risk Scoring Engine
attack_category	Category of the suspected attack
decision	Allow, additional check, or block
pattern_status	Approved or rejected status of any pattern proposed from the event
reviewer	Identity of the reviewer who handled the proposed pattern

Table 4.5 Attack Log Record Fields

4.5 TESTING STRATEGY

The proposed system follows a layered testing approach, where each module is tested individually before complete end-to-end testing is performed.

Test Category	Scope	Example Test Cases
Unit Testing	Individual layers, scoring formula, preprocessing steps	Known override phrase gives high L1/L2 score; harmless text gives low score
Integration Testing	Input firewall → risk engine → LLM → output firewall	Safe prompt reaches the LLM and a safe response is delivered
Security Testing	Evasion attempts, hidden instructions, filter abuse	Obfuscated override is still flagged; hidden instruction in a PDF is detected
Regression Testing	Re-running earlier cases after rule or model updates	Previously blocked attacks are still blocked after a lexicon update
Usability Testing	Block messages and reviewer interface	Blocked users receive a clear message; reviewers can act on candidates

Table 4.6 Testing Strategy Summary

Test data will include benign prompts, known attacks, paraphrased attacks, documents with and without hidden instructions, and responses with and without sensitive content. The planned functional test cases are listed in Table 4.7.

ID	Test Scenario	Expected Behaviour
TC-01	Submit an ordinary, harmless prompt	Low risk; request allowed and a normal response returned
TC-02	Submit a prompt with a known instruction-override formulation	High risk; request blocked and logged
TC-03	Submit a paraphrased manipulation attempt with no known keyword	Flagged by L3; medium or high risk
TC-04	Upload a clean PDF document	Document accepted and added to the safe index
TC-05	Upload a document containing hidden instructions	Instructions detected; content cleaned or document rejected
TC-06	Upload an image containing embedded text	Text extracted by OCR and screened
TC-07	LLM response contains sensitive data	Output firewall replaces the response with the safe default
TC-08	Reviewer approves a candidate pattern	Pattern added to L1/L2 and recorded in the audit log
TC-09	Reviewer rejects a candidate pattern	Pattern not deployed; rejection recorded

Table 4.7 Planned Functional Test Cases

4.6 DEPLOYMENT CONSIDERATIONS

The AI Firewall is designed as a separate layer around an LLM application, so it can be integrated in more than one way. Table 4.8 compares the three common integration styles. The modular design allows the same security modules to be used in each case, and in every option the on-device classifier and the data stores remain inside the organization’s own environment.

Option	Description	Advantages	Considerations
Embedded library	Modules run inside the application process	Lowest overhead; simple to deploy	Tighter coupling; updates need redeployment
Gateway / reverse proxy	All LLM requests pass through a separate firewall service	Central enforcement; one place to update rules	Adds a network hop; gateway must be highly available
Sidecar service	Firewall runs next to each application instance	Isolation; independent scaling and updates	More components to operate

Table 4.8 Deployment Options for the AI Firewall

For the current phase, the proposed implementation uses Python with a DistilBERT or MiniLM class classifier, a linear-time regex engine such as RE2, and a FAISS or ChromaDB vector index. Measures that keep the cost of checking low include running the fast layers first, invoking the classifier only when the policy requires it, precompiling patterns, screening documents once at upload time, and writing logs asynchronously.

4.7 LIMITATIONS OF THE CURRENT PHASE

The current phase has several limitations. The report presents the design and evaluation plan, and no measured detection or latency values are available yet. No detection system can guarantee that every attack is found, and adaptive attackers may probe the firewall until they find a gap. The weights and thresholds of the risk engine are configurable but must be calibrated on evaluation data, and the balance between false positives and false negatives may need to be re-tuned for each application.

The compact on-device classifier may be less capable than a large hosted model, and the lexicon and classifier are likely to be most effective for the languages and domains used in their preparation. The learning loop relies on human reviewers, which adds operational effort but prevents harmful patterns from being learned. These limitations are addressed in Chapter 5.

4.8 RESPONSIBLE USE AND PRIVACY CONSIDERATIONS

A security tool that inspects user content must itself be used responsibly. Logs contain user prompts, so the retention period and access rights should follow the privacy policy of the organization. Users should be told that their requests are screened and should receive a clear message when a request is blocked. The lexicon and classifier should be reviewed so that legitimate requests in different languages and writing styles are not blocked unfairly.

The review process keeps a person responsible for every change to the security rules, and on-device classification keeps request content within the organization’s environment. The project does not claim certification of any kind.

CHAPTER 5
CONCLUSION

The proposed AI Firewall combines multi-layer input filtering, an on-device AI classifier, risk-based decisions, secure RAG document screening, output filtering, and reviewed lexicon learning to protect LLM applications against prompt injection and model abuse. User inputs and documents are inspected before they can influence the model, and responses are inspected before they reach the user, providing defence in depth across the complete pipeline.

The keyword and regex filters provide fast detection of known attack indicators, while the on-device classifier provides deeper analysis of subtle, paraphrased, and potentially novel attacks without sending requests to an external service. The risk scoring engine combines these signals, and the continuous learning mechanism allows the firewall to improve from logged attacks while keeping a human reviewer in control.

Overall, the proposed approach addresses the identified research gaps and provides a practical, measurable framework for protecting LLM applications.

5.1 SUMMARY OF CONTRIBUTIONS

The main contributions of this project are: (i) a three-layer input pipeline that combines fast lexical checks with an on-device classifier; (ii) a weighted risk scoring method that produces graded security decisions; (iii) a RAG security module that screens documents before they enter the retrieval index; (iv) an output filtering firewall with a safe default response; and (v) a human-reviewed lexicon learning loop that improves the fast filters over time.

5.2 FUTURE WORK

Future work will focus on implementing the modules, integrating them into a single application, and testing them individually and through the complete end-to-end workflow. The weights and thresholds of the risk engine will be calibrated using representative data, and the system will be evaluated using the detection and latency metrics defined in this report.

Further extensions may include adaptive risk weights, detection of indirect prompt injection spread across several documents, a real-time security dashboard, integration of trusted threat-intelligence sources, privacy-preserving improvement of the classifier, support for multiple LLMs, and an automated benchmark that keeps testing the firewall against new attack scenarios.

REFERENCES

[1] OWASP Foundation. (n.d.). OWASP Top 10 for Large Language Model Applications. OWASP GenAI Security Project. https://genai.owasp.org

[2] Perez, F., & Ribeiro, I. (2022). Ignore previous prompt: Attack techniques for language models. arXiv:2211.09527.

[3] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. arXiv:2302.12173.

[4] Liu, Y., et al. (2023). Prompt injection attack against LLM-integrated applications. arXiv:2306.05499.

[5] Wei, A., Haghtalab, N., & Steinhardt, J. (2023). Jailbroken: How does LLM safety training fail? Advances in Neural Information Processing Systems (NeurIPS), 36.

[6] Zou, A., Wang, Z., Kolter, J. Z., & Fredrikson, M. (2023). Universal and transferable adversarial attacks on aligned language models. arXiv:2307.15043.

[7] Alon, G., & Kamfonas, M. (2023). Detecting language model attacks with perplexity. arXiv:2308.14132.

[8] Jain, N., et al. (2023). Baseline defenses for adversarial attacks against aligned language models. arXiv:2309.00614.

[9] Hines, K., et al. (2024). Defending against indirect prompt injection attacks with spotlighting. arXiv:2403.14720.

[10] Inan, H., et al. (2023). Llama Guard: LLM-based input-output safeguard for human-AI conversations. arXiv:2312.06674.

[11] Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems (NeurIPS), 33, 9459–9474.

[12] Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. arXiv:1910.01108.

[13] Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., & Zhou, M. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. Advances in Neural Information Processing Systems (NeurIPS), 33, 5776–5788.

[14] Johnson, J., Douze, M., & Jégou, H. (2021). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 7(3), 535–547.

APPENDIX

A. SAMPLE REQUEST SCREENING OUTPUT

The following examples illustrate the form of the screening result returned for a request. They are illustrative and are not measured results.

Allowed request:

✓ Input normalized ✓ L1 keyword score low ✓ L2 no pattern matched ✓ L3 score low ✓ Total risk below T_low **Status: ALLOWED**

Blocked request:

✓ Input normalized ✗ L1 keyword match ✗ L2 pattern matched ✗ L3 score high ✗ Total risk at or above T_high **Status: BLOCKED Reason:** High-risk request

B. SAMPLE ATTACK LOG RECORD

The following record illustrates how an event detected only by the classifier might be stored.

```
{
  "timestamp": "2026-10-04T10:15:30",
  "detection_layer": "L3",
  "attack_category": "paraphrased manipulation",
  "scores": { "L1": 0.00, "L2": 0.00, "L3": 0.90 },
  "risk_score": 0.45,
  "decision": "additional_check",
  "candidate_status": "queued_for_review"
}
```

C. SAMPLE CONFIGURATION FILE

The following listing shows how the parameters of Table A.1 could be stored. The values are placeholders used only to show the structure; they are not tuned results.

```
{
  "weights": { "w1": 0.2, "w2": 0.3, "w3": 0.5 },
  "thresholds": { "low": 0.35, "high": 0.70 },
  "classifier_policy": "when_unclear",
  "medium_risk_action": "additional_check",
  "fuzzy_tolerance": 1,
  "rag": { "ocr_enabled": true, "max_document_mb": 20 },
  "safe_default_response": "This request cannot be processed.",
  "reviewer_approval_required": true
}
```

Parameter	Description
W1, W2, W3	Weights applied to the L1, L2, and L3 scores
T_low, T_high	Thresholds separating low, medium, and high risk
Fuzzy-match tolerance	Spelling variation tolerated by the L1 keyword filter
Classifier invocation policy	Rule deciding when the L3 classifier is run
Medium-risk action	Additional checking or controlled allow
Maximum document size / OCR enabled	Limits and options used by the RAG Security Module
Safe default response	Message returned when a request is blocked or a response is unsafe
Reviewer approval required	Enforces human approval before any learned pattern is deployed

Table A.1 Configuration Parameters of the AI Firewall

D. SAMPLE FIREWALL PSEUDOCODE

The following pseudocode illustrates the structure of the request screening function referenced in Section 4.2.1.

```
function screenRequest(prompt, config):
    text = normalize(prompt)
    L1 = keywordScore(text, lexicon)
    L2 = regexScore(text, patterns)
    L3 = classifierScore(text) if needClassifier(L1, L2, config) else 0
    R  = w1*L1 + w2*L2 + w3*L3
    if R >= T_high:
        log(prompt, L1, L2, L3, R, "BLOCK"); return BLOCKED
    if R >= T_low:
        applyAdditionalChecks(prompt)
    response = LLM(prompt, retrieve(prompt, safeIndex))
    if outputUnsafe(response):
        log(prompt, response, "UNSAFE_OUTPUT"); return SAFE_DEFAULT
    return response
```

E. PROJECT RISK REGISTER

The following risks have been identified for the remaining phases of this project, along with the planned mitigation approach for each.

Risk	Likelihood	Mitigation
Thresholds set too strict, blocking legitimate requests	Medium	Review blocked-request logs during testing and adjust thresholds
Thresholds set too lenient, allowing attacks through	Medium	Evaluate on known and paraphrased attack sets and re-tune
Insufficient labelled test data	Medium	Combine public prompt-injection collections with hand-written examples
Compact classifier slower than expected on available hardware	Low	Invoke the classifier by policy; use batching or optional GPU
OCR errors cause hidden text to be missed	Medium	Test OCR on varied images; reject files that cannot be read
Harmful entries enter the lexicon	Low	Mandatory human approval and audit of every change

Table A.2 Project Risk Register

F. SAMPLE DECISION CODES AND MESSAGES

The following codes are returned by the firewall when a request, document, or response fails a check, allowing the front-end application to show a precise message instead of a generic failure notice.

Code	Triggering Condition	Example Message Shown to User
BLOCK_HIGH_RISK	Total risk at or above the high threshold	“This request cannot be processed.”
CHECK_MEDIUM_RISK	Total risk between the two thresholds	“Your request needs additional verification.”
DOC_REJECTED	Severe hidden instruction found in a document	“This document could not be added.”
OUTPUT_UNSAFE	Response failed the output firewall	“Sorry, I can’t help with that.”
CLASSIFIER_DEGRADED	On-device classifier unavailable	(No message; degraded mode recorded in the log)
OCR_FAILED	Text could not be read from an image	“This file could not be screened.”

Table A.3 Decision Codes

G. SAMPLE REVIEWER VIEW OF A CANDIDATE PATTERN

The following illustrates the information a reviewer is expected to see for a candidate pattern proposed by the lexicon learner. The values are illustrative.

Field	Displayed Value
Candidate ID	C-0001
Proposed type	Keyword phrase for L1
Source event	Event detected only by L3 (see Appendix B)
Supporting log entries	1 (example)
Available actions	Approve / Reject / Modify / Defer
Effect of approval	Added to L1 lexicon and deployed with the next filter update

Table A.4 Sample Reviewer View

The reviewer's decision is stored in the audit record together with the reviewer's identity and the time of the decision.

H. LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
DFD	Data Flow Diagram
FAISS	Facebook AI Similarity Search
LLM	Large Language Model
OCR	Optical Character Recognition
PII	Personally Identifiable Information
RAG	Retrieval-Augmented Generation
REC	Rajalakshmi Engineering College
SDG	Sustainable Development Goal
UML	Unified Modeling Language